

Data Preparation Session 3 Handout

Data structure – long and wide data

After importing the data, the next step is to check if the data is in the right structure. Data is not always tidy, we start with rows, columns mixed up. In this session, we focus on large, structural changes.

A dataset can be represented in several formats, with more rows (and less columns), or with less rows (and more columns); The first one is called long, the second one is called wide data. Both of them can be useful for data analysis, however, in many cases, long data fits better to data analysis tasks.

How to make your data longer or wider?

Using the `pivot_longer()` and `pivot_wider()` functions (previously `gather()` and `spread()`).

```
pivot_longer(data, column(s), names_to, values_to)
```

```
pivot_wider(data, names_from, values_from)
```

- name is to identify the variable described by value
- value is the data value that represents an observation of a single variable

Let's start with as simple problem.

```
pew <- read_csv("pew.csv")
```

The dataset is ready for the analysis of the relationship of religion and income, a classical cross-tabulation.
`pew`

It is perfect when we work in Excel, and we want to realize a Chi-square test, but typically not optimal for general data analysis purposes.

The problem? A variable is stored as variable names. The income should be a variable.

We have to make it longer. Look at the arguments of the `pivot_longer()` function, first, the data, followed by the columns to make longer (can be a positive or a negative list), `names_to` means what will be the new variable created from the the current variable names, `value_to` stands for the new variable where we put all current data values. Pay attention to variable names, they are strings, as they are not existing objects yet.

```
pew.long <- pivot_longer(pew, !religion, names_to='income', values_to='freq')
```

```
pew.long
```

Still, we don't have the individuals, but at least each variable is really a variable.

Another simple example is the Mexican weather dataset.

```
weather <- read_csv("mexicanweather.csv")  
glimpse(weather)
```

The problem is with the "element" column, as there are two variables, the minimum and the maximum temperatures. We should "pull it up" and sort the values into two columns, in other words, we have to make it wider.

Using the `pivot_wider` function, we need the first argument as the data, `names_from` stands for the variable that stores many variables, and `values_from` for the values in the original data. As these variables exist in the dataset, no quotation mark is needed.

```
weather.wide <- pivot_wider(weather, names_from=element, values_from=value)
```

```
weather.wide
```

The structural problem is solved, however there are a few remaining problems with the dataset.

Let's see a third example, here we will see additional arguments of the pivot functions.

We read a txt file. We have to figure out the file type. You can check the file in a simple text editor, it can be either tab separated or fix width (however, this last one does not work for the Bond column).

```
bond_df <- read_tsv('bond.txt')
```

The problem is that we have a variable in the variable names (the decade). Thereby, it is a wide dataset, and we should make it long.

```
bond_long <- pivot_longer(bond_df, -Bond, names_to="decade", values_to = "n_movies")
```

We have a few problems left. First, there are a lot of NAs. NA stands for missing value, in this phase, we can leave it as we found, or we can decide to remove them, only keeping useful information; Alternatively, we can replace them by 0 (see it later).

```
glimpse(bond_long)
```

We can see that the data type of the decade is chr, so a text(string) variable. It happens when we transform variable names to values, and by the principles, a variable name cannot be a number. We can leave it as it is, or we can solve it here. The `names_transform` argument needs a list, even if it is a one-element list.

A better version of the transformation:

```
bond_long <- pivot_longer(bond_df,
  -Bond,
  names_to = "decade",
  values_to = "n_movies",
  values_drop_na = TRUE,
  names_transform = list(decade = as.integer)
)
```

We move now to a complex problem. In this problem, we will see a transposed dataset, rows are columns and columns are rows. There are many ways how we can solve the problem. Here, we will see with two basic pivot manipulations, however, transforming the data into a chr matrix, using `t()` and defining data types is also possible.

We read the data. As you remember, the data can be found on the second worksheet of the Excel workbook.

```
library(readxl)
planet_df <- read_excel("planets.xlsx", sheet = 2)
planet_df
```

A `pivot_longer()` transform the data into 3 columns, keeping all information.

```
planet_long <- planet_df %>%  
  pivot_longer(-metric, names_to = "planet")
```

In this version, we see that the metric column is not a variable, but it stores a series of variable names, and the value column stores mixed values, it is also not a variable. We make it wide:

```
planet_wide <- planet_long %>%  
  pivot_wider(names_from = metric, values_from = value)
```

Or, in one step:

```
planet_df <- planet_df %>%  
  pivot_longer(-metric, names_to = "planet") %>%  
  pivot_wider(names_from = metric, values_from = value)  
planet_df
```

Typically, we have to deal with variable types. Why? As a principle, if in a variable we have at least one string, it is a string, and a string variable sorted into many variables remains string.

We can use the `mutate()` function, and we change the numeric variables to numeric or double type (both will appear as *dbl* in `glimpse`, and as *num* in the environment).

```
planet_df <- planet_df %>%  
  mutate(mass = as.numeric(mass),  
         diameter = as.double(diameter),  
         gravity = as.double(gravity),  
         length_of_day = as.double(length_of_day),  
         distance_from_sun = as.double(distance_from_sun),  
         mean_temperature = as.double(mean_temperature),  
         surface_pressure = as.double(surface_pressure),  
         number_of_moons = as.integer(number_of_moons)  
  )  
planet_df  
glimpse(planet_df)
```

An alternative, higher level solution using `mutate_at()`:

```
planet_df <- planet_df %>%  
  mutate_at(vars(2:8), funs(as.double)) %>%  
  mutate_at(vars(number_of_moons), funs(as.integer))  
glimpse(planet_df)
```

Or another alternative (most up-to-date):

```
planet_df <- planet_df %>%  
  mutate(across(2:8, as.double)) %>%  
  mutate(across(number_of_moons, as.integer))  
glimpse(planet_df)
```